PLP Admissions System — Full Flow & Reference

End-to-end behaviour, scoping rules, edge cases, and codebase map

Verified against `baseline` after the latest project zip merge

> Verified against the code on `baseline` (post-zip-merge, post-role-redesign). When something in this document disagrees with the code, **the code is the source of truth** — please open a PR to update this file.

Table of Contents

1. System Overview

PLP Admissions is a vanilla-PHP web app for Pamantasan ng Lungsod ng Pasig that runs the entire student admissions cycle — registration → email verification → document review → entrance exam → interview → admission decision → enrollment intent.

Stack: PHP 8 (no framework), MySQL / MariaDB (utf8mb4), vanilla CSS / JS, PHPMailer (SMTP), hCaptcha (anti-bot), Chart.js for dashboards.

Architecture: one entry point (`public/index.php`) routes every request through a hand-rolled Router. Each feature lives in `modules/<area>/` and renders into a shared layout via `ob_start()` / `ob_get_clean()`. Cross-cutting logic (auth, automation, the interview scheduler) lives in `core/`.

2. Roles & Access

The seeded SQL provisions **six** role tiers. `Auth::homeUrl()` in `core/Auth.php` decides where each role lands after login.

Role	Default landing page	What they do
Admin	`/admin/dashboard`	System-wide oversight: users, school year window, courses & caps, branding, settings, audit log, all reports, Auto Release & Close Admissions.
SSO	`/admin/dashboard`	Office of Student Services. Reviews documents, builds the exam, sets exam & interview slots, runs Cancel & Move workflows. **No access to Results.**
Dean	`/admin/dashboard`	Per-college oversight: sees applicants in their college, edits course caps + tier thresholds, **releases Results for their applicants**.
Staff (Professor)	`/staff/dashboard`	Per-college interviewer. Runs their own live queue and records a Pass / Decline recommendation per applicant.
Proctor	`/staff/dashboard`	Exam-day operations. Generates 6-character access codes for their own college's exam rooms.
Student	`/student/documents`	Applicant. Uploads documents, takes the exam, sees their interview slot, confirms enrollment.

The seeded passwords (from `database/seed_users.sql`) follow the `Role@123` pattern (`Admin@123`, `SSO@123`, `Dean@123`, `Staff@123`, `Proctor@123`). **Change them before going to production.**

Role redesign (vs. earlier builds)

- Result-release authority **moved from SSO → Dean**. SSO is locked out of `/staff/results`; the auth gate is `Auth::requireRole(ROLE_DEAN, ROLE_ADMIN)`.
- Interview evaluation outcomes are now **Pass / Decline** (stored internally as `pass` / `reject`, not `pass` / `fail`).
- New **Proctor** role with a thin sidebar (Dashboard + Exam Slots only).

---

3. The Admissions Pipeline (end to end)

Every applicant moves through a single column on the `applicants` table: `overall_status`. The values, in order, are:

```
pending → documents → submitted → exam → interview → released
                                                       ↘
                                                    withdrawn (terminal)
```

Below is what triggers each transition.

Phase 0 — Admin / SSO configuration (before admissions open)

These steps happen once per cycle. None of them touch student data.

1. **Admin opens the admissions window** (`/admin/school-year`):
   - Sets the open date, close date, and optional document submission deadline.
   - The current school year is auto-derived (e.g. opening in 2026 → `2026-2027`).
   - Outside this window, `/register` is blocked with a friendly message.
2. **Admin / Dean configures courses, strand maps, and caps** (`/admin/courses` `/admin/courses`):
   - Courses live in `course_departments`. Each course has an `avg_from` rank (1–10, the Average / passing threshold) and a `high_from` rank (1–10, the High / top-tier threshold).
   - `pass_from` is derived from `avg_from` in code.
   - Each course has an optional cap (`course_caps.max_slots`).
   - Caps are enforced at registration time and at result-release time.
3. **Admin provisions Staff / Dean / SSO / Proctor accounts** (`/admin/users`):
   - Every Staff / Dean / Proctor account must have a `department` matching the college it serves (e.g. College of Computer Studies). This is what scopes the interview queue, results, and Proctor's exam slot view.
4. **SSO builds the entrance exam** (`/staff/exam`):
   - Title (auto-derived as `PLP Admissions Test ({school_year})`), description, shuffle toggles, sections, questions (multiple choice, checkbox, dropdown, short answer, paragraph, linear scale).
   - Only one exam can be active at a time (`is_active = 1`).
5. **SSO creates exam rooms** (`/staff/exam/slots`):
   - Date + per-slot opens / closes time + room label + department + capacity. Default capacity is 35; daily department cap is 3000.
   - **Batch Create** stamps multiple rooms on the same day.
6. **Staff / SSO creates interview sessions** (`/staff/interviews/setup`):
   - Per-college list. **+ Add Session** asks for date, start / end time, capacity, assigned interviewer, location label, location notes. There is no separate Desk concept — desks and sessions were merged into `interview_slots`.

Phase 1 — Registration

`POST /register` (`modules/auth/register.php`):

1. **Admissions-window check.** Blocked if closed.
2. **hCaptcha verification.**
3. **Field validation:** name, birthdate, sex, street address + Pasig barangay (hardcoded list of 30), phone, email, password (≥ 8 chars), applicant type (`freshman` | `transferee` | `foreign`), course, SHS strand (freshmen only).
4. **Course-cap pre-check** — if `accepted_count ≥ max_slots` for the chosen course in the current school year, registration is blocked with "This course has reached its enrollment cap…". Capped courses are also hidden / disabled in the registration dropdown.
5. **Strand compatibility check** — a freshman's SHS strand must be in the course's strands allowlist.
6. **Duplicate detection** — same `first_name + last_name + birthdate` is rejected.
7. **On success, inside a transaction:**
   - Insert into `users` with `role = 'student'`, `department = course_to_department(course)`, password bcrypt-hashed (cost 12).
   - Insert into `applicants` with `overall_status = 'pending'`.
   - Pre-create the required `documents` rows in `status = 'pending'`.
8. **Outside the transaction:**
   - Generate a verification credential pair (magic-link token + a 6-digit code).
   - Send the verification email via PHPMailer + Gmail SMTP.
   - The user is NOT auto-logged in. They're redirected to `/verify-pending`.

Phase 2 — Email verification

`/verify-pending` (`modules/auth/verify_pending.php`):

- Shows a 6-digit code form and a "Resend code" button with a cooldown (default 60 seconds).
- Code verification: success → auto-login + redirect to `/student/documents`. Failure increments `email_verify_attempts`.
- The magic link in the email (`/verify-email?token=…`) does the same thing — clicking it logs the user in.
- `modules/auth/login.php` redirects unverified students back to `/verify-pending` rather than refusing them.

The verification columns (`email_verified`, `email_verify_token`, `email_verify_code`, `email_verify_code_expires_at`, `email_verify_attempts`, `email_verify_last_sent_at`) are auto-created by `ensure_email_verification_columns()` in `core/automation.php`.

> **Demo bypass:** `database/seed_demo.sql` inserts every demo student with `email_verified = 1` so they skip the verification gate during presentations.

Phase 3 — Document submission

`modules/documents/student_upload.php`:

1. **Document list** depends on applicant type (from `config/app.php`):
   - **Core (all):** government ID, PSA birth certificate, passport photos, parent ID, proof of income, guardianship affidavit.
   - **Freshman:** Form 138 (or Form 137).
   - **Transferee:** TOR + good moral.
   - **Foreign:** TOR + good moral + passport + visa / study permit + alien certificate.
2. **Upload constraints:** PDF / JPG / PNG / WEBP, ≤ 5 MB per file.
3. **Basic validation** at upload: MIME-type + size + integrity check. The OCR-style auto-validation pipeline and the `api/auto-validate` endpoint have been **retired in this build**.
4. **Status transitions:**
   - Upload moves a single document from `pending` → `uploaded`.
   - Once all required documents are uploaded (or already approved) the student can click **Submit**, which sets `applicants.overall_status = 'submitted'`.
   - Student can withdraw their submission (back to `documents`) up until the first staff approval.
   - Once `overall_status` is past `documents` (i.e. `exam`, `interview`, `released`), changing applicant type is locked.
5. **Document deadline enforcement** — if the admin set a doc deadline and it has passed, applicants who haven't submitted see a "Document Submission Closed" page. POST is blocked server-side. Applicants who already submitted continue normally.

Phase 4 — Staff / SSO document review

`modules/documents/staff_review.php` is the document-review queue, used by SSO (and Admin / Dean as oversight).

- Default tab: applicants with `overall_status IN ('documents', 'submitted')`.
- **Per-document actions:**
  - **Approve** — accept the document.
  - **Request Resubmission** — softer alternative; sets the doc back to `rejected` with `staff_remarks`. The student sees the reason and can re-upload. **There is no plain "Reject" button.**

- **Bulk actions:** Approve Selected, Approve All Pending Reviews.

The moment every required document is approved, `staff_action.php` runs:

```
UPDATE applicants
   SET overall_status = 'exam',
       documents_approved_at = COALESCE(documents_approved_at, NOW())
 WHERE id = ?
   AND overall_status NOT IN ('exam','interview','released')
```

Then it calls `notify_stage_transition()` (in-app + email) and `auto_assign_exam_slot()`.

Undo approval is allowed only while the applicant hasn't taken the exam yet. It rolls `overall_status` back from `exam` to `submitted`.

Phase 5 — Entrance exam

Once `overall_status = 'exam'`, `auto_assign_exam_slot()` (`core/automation.php`) picks the earliest-available, lowest-fill exam room in the applicant's department and inserts a row into `applicant_exam_slots`. SSO can also assign manually from `/staff/exam/slots`.

After adding a new slot, `staff_slots.php` silently re-runs `backfill_exam_slot_assignments()` so any waiting applicants in the department are placed into the new slot (the success flash reports the count).

`modules/exam/take.php` (the student-facing exam page) renders in three states:

1. No slot yet → "Awaiting Slot Assignment" notice.
2. Slot is in the future → countdown card with date, time, room.
3. Slot is today → access-password gate, then the exam itself.

Access code: `generate_exam_password()` returns a **6-character** string of uppercase letters + digits with ambiguous characters (`0`, `O`, `1`, `I`, `L`) excluded. Codes are valid for `EXAM_PASSWORD_EXPIRY_SECONDS = 300` (5 minutes). **Extend** keeps the same code and resets the timer; **New** issues a fresh code and invalidates the previous one. Both actions are audited as `exam_slot_code_extended` and `exam_slot_code_generated`.

Anti-cheating measures during the exam: text selection disabled; timer counts from scheduled start to end time; autosave every 60 s to `/api/exam-autosave` → `exam_drafts`; on reload, drafts are restored.

On submit:

```
raw_score   = sum of correct points across auto-gradable items
percentage  = raw_score / total * 100
rank        = ceil(percentage / 10)            # clamped 1..10
passed      = rank >= course_passing_scores.pass_from  # default 4
```

Tier labels (rendered on the result page):

Rank	Tier	Verdict
7–10	High	Passed
4–6	Average	Passed
1–3	Low	Rejected

- **Passed:** `overall_status` flips to `interview`, and `assign_interview_slot()` is called immediately (see Phase 6).
- **Failed:** status stays at `exam`. `suggest_alt_courses()` proposes courses with a lower `pass_from` that the score would have qualified for; SSO / Dean / Admin can then push a suggestion via the course-suggestion flow in `staff_suggest.php`. The student sees the suggestion on `/student/result` and can Accept (which switches `course_applied` and rolls them back into the pipeline) or Decline.

Phase 5.5 — Exam reschedule (NEW)

`modules/exam/student_reschedule.php` exposes a **Request Reschedule** button on `/student/exam`. It posts to `POST /api/exam-reschedule-request`:

1. Validates that the student is currently at the `exam` stage with a current `applicant_exam_slots` row.
2. Inserts a `pending` row into `exam_reschedule_requests` with the student's reason.
3. Notifies SSO / Proctor / Dean / Admin (in-app + email).

`/staff/exam/reschedule` lets SSO / Admin view pending requests, accept (replacement-slot dropdown or auto-assign earliest open slot), or deny with a reason. Dean and Proctor see it read-only.

Phase 5.6 — Exam slot bulk cancel & move (NEW)

`/staff/exam/cancel-slot` (SSO + Admin):

1. Pick the slot to cancel.
2. Pick a replacement slot with `(capacity − booked) ≥ cancelled.booked`.
3. Enter a written reason.
4. Submit.

The handler locks rows in a `FOR UPDATE` transaction, re-validates capacity inside the transaction, moves every applicant in the cancelled slot to the replacement, and fires per-student in-app notifications and branded emails carrying the reason.

Phase 6 — Interview

`core/interview_scheduler.php :: assign_interview_slot()`:

1. Resolves the applicant's department from `users.department` first, falling back to `course_to_department(course_applied)` and opportunistically backfilling `users.department`.
2. Inside a `FOR UPDATE` transaction:
   - Lock all open future slots in that department.
   - Pick the one with the lowest `booked` count → earliest `slot_date` → earliest `slot_time` (fair distribution).
   - Refuse if the applicant already has an active queue row (UNIQUE constraint on `interview_queue.applicant_id`).
3. Insert into `interview_queue`:
   - `status = 'checked_in'` (yes, immediately — there is no longer an "I'm Here" button).
   - `queue_number = next sequential per slot`.
   - `checked_in_at = NOW()`.
4. Audit log + in-app + email notification.

If no slot exists yet (e.g. nobody has created sessions for that college), the applicant stays at `overall_status = 'interview'` with no queue row. The next time a Staff member creates a session for that department, `bulk_assign_pending_applicants($dept)` sweeps every waiting applicant into the new slot.

On interview day (`modules/interview/staff_queue.php`):

- The Live Queue page is scoped:
  - **Staff (Professor)** → only rows where `COALESCE(s.assigned_to, s.created_by) = self`.
  - **Dean** → all rows where `s.department = staff.department` (read-only on the queue itself; Dean does not evaluate).
  - **Admin / SSO** → can pick a college or see everything via `?college=__all__`.
- **Actions:**
  - **Call Next** flips the next `checked_in` row to `in_progress`.
  - **Evaluate** records `evaluation_result` (Pass / Decline, stored as `pass` / `reject`) on the queue row, sets `interview_completed_at` on the applicant, AND immediately flips `overall_status` to `released` so the applicant shows up on the Results page in the matching Recommended bucket.
  - **Mark Absent** sets `attendance_status = 'absent'`. Auto-reschedule (`auto_reschedule_noshows`) can route them to the next available slot.

- **Auto no-show**: every page load runs an UPDATE that flips any still-waiting / in-progress row past its end time to `no_show` + `absent`. There is no manual "No-show" button.

The applicant page (`/student/interview`) is read-only — it shows their date, time, location, interviewer, and a live queue position computed against `checked_in` rows ahead of them. Students can submit a reschedule request from the same page (POST to `/api/reschedule-request`).

Phase 6.5 — Interview slot bulk cancel & move (NEW)

`/staff/interviews/cancel-slot` (SSO + Dean + Admin) mirrors the exam-side flow. Same locking, same notifications.

Phase 7 — Admission decision (Results)

`modules/results/staff_manage.php` is the Results console. **Auth gate:** `Auth::requireRole(ROLE_DEAN, ROLE_ADMIN)`. SSO is **locked out** in the role redesign.

It buckets applicants:

Bucket	Label in UI	Predicate
`awaiting`	Awaiting interview	Interview not yet evaluated.
`ready_accept`	**Recommended: Accept**	`exam_passed = 1 AND interview = pass`
`ready_reject`	**Recommended: Decline**	`exam_passed = 0 OR interview = reject`
`released`	Released	Already has an `admission_results` row.
`withdrawn`	Withdrawn	`applicants.overall_status = 'withdrawn'`

> **Note on waitlist:** `admission_results.result` still allows `waitlisted` for backward compatibility, but the current staff UI only emits `accepted` or `rejected`. `auto_promote_waitlist()` in `core/automation.php` is a documented no-op stub.

Per-row buttons:

- **Accept** (green) — releases as `accepted`. Matches the recommendation in the Accept bucket; in the Decline bucket it overrides and pops the override-reason modal.
- **Reject** (red) — releases as `rejected`. Matches the recommendation in the Decline bucket; in the Accept bucket it overrides and pops the override-reason modal.

Overrides require a non-empty written reason, which is stored on `admission_results.remarks` and written to the audit log as `admission_result_override`.

Toolbar actions:

- **Bulk Accept** / **Bulk Reject** (Dean + Admin) — checkbox selection + Accept Selected / Reject Selected. Calls `staff_bulk.php`. Skips withdrawn applicants and applicants with an existing result.
- **Auto Release** (Admin-only) — `POST /staff/results/auto-release` calls `auto_release_results()`, which walks every `overall_status IN ('exam','interview','released')` row without a result and emits `accepted` (exam passed AND interview passed) or `rejected` (exam failed OR interview failed). Skips applicants whose interview hasn't been evaluated yet. Only runs when `school_settings.auto_release_results = '1'`.
- **Close Admissions** (Admin-only, red) — opens a confirmation modal that demands a reason. On submit, every applicant in the current cycle that isn't already Released or Withdrawn is bulk-rejected with the supplied reason. Irreversible.
- **Suggest Alternative Course** (Dean / Admin) — for failed-exam applicants who still qualify for a different course. Validates the applicant's rank against the suggested course's `pass_from`, then upserts a `course_suggestions` row that the student sees on their result page.

The release transaction is:

1. `BEGIN`.
2. `INSERT INTO admission_results (applicant_id, result, remarks, released_by, released_at) VALUES (…)` — UNIQUE on `applicant_id` means a second concurrent insert raises `23000` and rolls back.
3. `UPDATE applicants SET overall_status = 'released' WHERE id = ?`.
4. `COMMIT`.
5. Send in-app + email notification.
6. Audit log row.

Bulk release (`staff_bulk.php`) does the same in a loop, one transaction per applicant, so a single failure doesn't roll back the entire batch.

Phase 8 — Enrollment intent

`modules/results/enrollment_intent.php` handles `POST /student/result`:

- Accepted students see "I Confirm My Enrollment" and "Decline Slot" on on `/student/result`.
- **Confirming** sets `admission_results.enrollment_intent = 'confirmed'` and stamps `intent_submitted_at`.

- **Declining** sets `enrollment_intent = 'declined'` and flips `overall_status = 'withdrawn'`.
- Withdrawing also accepts a free-text reason saved to `applicants.withdrawn_reason`.

There is also an auto-expire sweep (`auto_expire_accepted_pending` in `automation.php`): accepted students who don't act within `enrollment_intent_deadline_days` (default 7) get auto-withdrawn with a "Slot expired" notification.

---

4. Automations Cheat-Sheet

Every automation toggle lives in `school_settings` and is toggleable from `/admin/settings`.

Setting key	Default	What it does
`auto_assign_exam_slots`	1	Drop applicant into the next exam room when all docs are approved.
`auto_reschedule_noshows`	1	Move interview no-shows to the next available slot for their department.
`auto_release_results`	0	Allow the auto-release sweep to flip Recommended: Accept / Recommended: Decline → released.
`auto_promote_waitlist`	1	Deprecated — the function is a no-op stub since waitlist was retired.

Triggers that always fire (not toggleable):

- All-docs-approved → `overall_status = 'exam'` + `auto_assign_exam_slot()` + notification.
- New exam slot created → `backfill_exam_slot_assignments()` sweeps waiting applicants in that department into the new slot.
- Exam pass → `overall_status = 'interview'` + `assign_interview_slot()` + notification.
- Staff creates a new interview session → `bulk_assign_pending_applicants()` sweeps unscheduled applicants in that department into the new slot.
- `interview_queue` evaluation → applicant `overall_status` flips to `released`; if Pass, they land in Recommended: Accept; if Decline, Recommended: Decline. Nothing is auto-released unless the toggle is on.
- Notifications (in-app + email via PHPMailer) on every status transition.
- Audit log row on every state-changing action.

5. Department Scoping (who sees whom)

This is the area that caused the demo seed regression — worth calling out explicitly.

Page	Scope rule
`/student/documents`	Always shows the logged-in student's own applicant. No cross-applicant access.
`/staff/applicants` (doc review)	Unscoped by college — this is the SSO global doc-review queue. SSO, Dean, Admin all see all colleges here.
`/staff/applicants/{id}`	Permitted for SSO / Admin always. Dean is granted only if `users.department = applicant.department`.
`/staff/exam`	SSO + Admin only.
`/staff/exam/slots`	Staff + Dean → their own college. SSO / Admin → college picker. Proctor → their own college only (no college selector).
`/staff/exam/reschedule`	SSO / Admin write; Dean / Proctor read-only.
`/staff/exam/cancel-slot`	SSO + Admin.
`/staff/interviews/setup`	Staff + Dean see their own college's sessions. Admin / SSO can pick a college.
`/staff/interviews/queue`	Staff → only their own assigned / created sessions. Dean → all sessions in their college (read-only). Admin / SSO → college picker.
`/staff/interviews/cancel-slot`	SSO + Dean + Admin.
`/staff/results`	Dean + Admin only. SSO is **locked out**. Dean is dept-scoped (sees their college only); Admin sees all.
`/staff/results/close` (Close Admissions)	Admin only.
`/staff/results/auto-release`	Admin only.
`/admin/users`, `/admin/school-year`, `/admin/settings`	Admin only.
`/admin/courses`	Admin → all courses. Dean → can edit `max_slots` + tier thresholds on courses in their college.
`/admin/audit-log`	Admin only.

6. Edge Cases & Business Rules

6.1 Course is full at registration

Cap check runs against `course_caps.max_slots` vs current accepted count. Registration is blocked with a clear error and the course gets a red “Full” badge in the UI.

6.2 Student fails the exam

Stays at `overall_status = 'exam'`. `suggest_alt_courses()` lists courses with a lower `pass_from` that the score would have qualified for. SSO / Dean / Admin can suggest one via `/staff/results/suggest/{id}`. The student sees the suggestion on `/student/result` and can Accept (which changes their `course_applied`) or Decline.

6.3 Document rejection / resubmission

Rejecting a document does **not** roll back the applicant if they were already past `documents`. **Request Resubmission** is the only path back — it puts the document back to `rejected` with staff instructions in `staff_remarks`. Either way the student sees the reason and can re-upload.

6.4 Interview no-show

Every page load on the queue auto-flips waiting / in-progress rows past their slot’s end time to `no_show` + `absent`. Staff can also hit **Mark Absent** manually. If `auto_reschedule_noshows = 1`, the next sweep books them into the next available slot for their department.

6.5 Student reschedule request

- **Exam reschedule:** `/student/exam` shows a Request Reschedule button → POST button → POST `/api/exam-reschedule-request` → row in `exam_reschedule_requests` → reviewed at `/staff/exam/reschedule`.
- **Interview reschedule:** `/student/interview` shows a “Need to reschedule?” details panel → POST `/api/reschedule-request` → row in `reschedule_requests` → reviewed at `/staff/interviews/absent?tab=requests`.

6.6 Bulk slot cancel & move

When an emergency (typhoon, room closure) forces a slot to be cancelled, SSO / Admin (exam) or SSO / Dean / Admin (interview) opens the corresponding **Cancel & Move** page, picks a replacement slot, enters a reason, and submits. The handler locks rows in a `FOR UPDATE` transaction, re-validates capacity, moves every applicant, and fires notifications.

6.7 Admissions window closed

`/register` shows a “Closed” page. Existing applicants are unaffected and can still log in to continue their journey.

6.8 Document deadline passed

Students who haven’t yet submitted see “Document Submission Closed”. POST to upload / submit is blocked server-side. Submitted students proceed normally.

6.9 Close Admissions (Admin)

Admin opens `/staff/results` and clicks **Close Admissions** → modal demands a reason → confirms. Every applicant in the current cycle that isn’t already Released or Withdrawn is bulk-rejected with the supplied reason recorded per row. Irreversible.

6.10 Withdrawal

Allowed at any stage before `withdrawn`. Sets `overall_status = 'withdrawn'`, `withdrawn_at = NOW()`, optional reason. The student loses their interview slot and admission result row.

6.11 Acceptance expired (no enrollment intent)

After `enrollment_intent_deadline_days` (default 7), accepted applicants who didn’t confirm or decline are auto-withdrawn by `auto_expire_accepted_pending()` with a “Slot expired” notification.

6.12 Staff undoes a document approval

Allowed only if the applicant hasn’t taken the exam yet. Rolls `documents.status` from `approved` to `uploaded` and `applicants.overall_status` from `exam` to `submitted`.

6.13 Custom courses

Admin can add courses beyond the built-in PLP programs via `/admin/courses`. Each custom course gets its own strand allowlist, `avg_from` / `high_from` thresholds, and `max_slots`. They appear in registration immediately.

6.14 Session timeouts

Student sessions expire after 30 minutes of inactivity; staff sessions after 2 hours. A warning appears 5 minutes before expiry. `POST /auth/keepalive` extends the session.

6.15 Override-reason audit

Every time the Dean's release decision contradicts the Professor's recommendation, a non-empty written reason is required and is stored on `admission_results.remarks` and logged as `admission_result_override` in `audit_logs`.

---

7. Frequently Asked Questions

"Can a CCS professor interview a CON student?" No (by default). Live-queue scoping for Staff is `COALESCE(s.assigned_to, s.created_by) = me` — and a Staff member can only create sessions in their own department, so they will never end up assigned to a CON session. An Admin / SSO can override by explicitly assigning across colleges, but that's a deliberate action, not the default.

"What if someone uploads a fake document?" At submit time the system verifies file type, size, and integrity. Authenticity verification is human-driven — staff reviews each document manually. The OCR-style auto-validate pipeline was removed in this build.

"What prevents impersonation at the exam?" Login + per-slot access code + name shown on the exam interface. There's no facial verification or proctoring software. In-room Proctors should still check physical IDs.

"What happens if the server dies mid-exam?" Answers autosave to `exam_drafts` every 60 s. On reload after recovery, drafts are restored.

"Can the Dean overturn an interview recommendation?" Yes — that's the whole point of the override flow. The Dean clicks Accept on a Recommended: Decline row (or Reject on a Recommended: Accept row); the modal demands a written reason; the reason is stored on `admission_results.remarks` and logged as `admission_result_override`.

"Can SSO release results?" No — not in this build. SSO is locked out of `/staff/results`. The auth gate is `Auth::requireRole(ROLE_DEAN, ROLE_ADMIN)`.

"Can staff manipulate results?" Every state change writes to `audit_logs` (actor user, IP, action, entity, timestamp). The admin audit log at `/admin/audit-log` shows the full trail. Logs are stored in the same DB as the data, so an admin with raw DB access could tamper — for high-stakes deployments, export the log to an external store.

"How many applicants can it handle?" Realistically, single-institution scale (a few thousand applicants per cycle). MySQL pagination is everywhere, queries are indexed, exam slots have configurable capacity. The exam-submit path is the hot spot.

"Can we run two school years simultaneously?" No. `current_school_year` is a single global setting; only one exam can be `is_active = 1`.

“What data can we export?” CSV from `/admin/dashboard` (applicants with filters) and `/admin/results`. No PDF letters yet.

“Can parents log in?” No — there’s no guardian portal. Only the student account.

---

8. Known Gaps & Risks

#	Gap	Severity	Status
1	Audit logs are not append-only at the DB level — an admin with SQL access can edit them.	Medium	Open
2	No 2FA for staff / admin.	Medium	Open
3	No PDF report generation (admission letters, exam summaries).	Low	Open
4	Exam UI is laptop-oriented; mobile layout is rough.	Low	Open
5	No backup / restore docs — `schema.sql` is destructive.	Medium	Open
6	No accessibility (ARIA / screen-reader) pass.	Low	Open
7	Rejected students can’t reapply in a future cycle without admin intervention.	Low	Open
8	No support for concurrent school years.	Low	By design
9	Waitlist tier is half-retired — schema still allows it but the UI doesn’t emit it.	Low	Tech debt
10	OCR-style auto-validation pipeline was removed; documents now rely entirely on human review.	Low	By design

Already addressed in earlier work (kept here for historical context):

- Email verification (verification email + code form).
- Duplicate applicant detection on registration.
- Exam autosave (`exam_drafts`).
- Student-initiated reschedule requests (exam + interview).
- Bulk Cancel & Move (exam + interview).
- Applicant type change before submission.
- Login rate limiting (15-minute lockout after 5 failed attempts).
- CSP / X-Frame-Options / X-Content-Type-Options security headers.
- Email notifications via PHPMailer + Gmail SMTP.
- Enrollment-intent flow (confirm / decline).

- Override-reason gate on result releases.
- Close Admissions bulk-reject (Admin).
- Secrets in `.env`, not in source.

---

9. Codebase Map

Core infrastructure

File	Purpose
`config/app.php`	Constants: paths, role names, document slugs per type, course list, strand
`config/db.php`	PDO MySQL connection, SSL support for cloud DBs.
`core/bootstrap.php`	Loads config, session, auth, router, helpers, automation in order.
`core/Auth.php`	Login / logout / role guards / `homeUrl()`.
`core/Session.php`	Session lifecycle + flash messages.
`core/Router.php`	Path routing including `/staff/applicants/{id}` style params.
`core/helpers.php`	URL / CSRF helpers, admissions window, `score_to_rank`, `exam_passed`, `generate_exam_password`, etc.
`core/automation.php`	All the auto-* logic: notifications, exam slot assignment, results auto-releas
`core/interview_scheduler.php`	Interview-side algorithms: `assign_interview_slot`, `bulk_assign_pendin` `reschedule_absent_applicant`.

Authentication

Path	File
`/login`	`modules/auth/login.php`
`/register`	`modules/auth/register.php`
`/verify-email`	`modules/auth/verify_email.php` (magic-link path)
`/verify-pending`	`modules/auth/verify_pending.php` (6-digit code)
`/forgot-password`, `/reset-password`	`modules/auth/forgot_password.php`, `modules/auth/reset_password.php`
`/auth/keepalive`	`modules/auth/keepalive.php`
`/logout`	`modules/auth/logout.php`

Student-facing

Path	File
`/student/documents`	`modules/documents/student_upload.php`
`/student/exam`	`modules/exam/take.php` (+ `student_reschedule.php` request)
`/student/interview`	`modules/interview/student_view.php`
`/student/result` (GET)	`modules/results/student_view.php`
`/student/result` (POST)	`modules/results/enrollment_intent.php`
`/student/settings`	`modules/settings/student.php`

Staff / Dean / SSO / Proctor

Path	File
`/staff/dashboard`	`modules/auth/staff/dashboard.php`
`/staff/applicants` (queue + per-applicant)	`modules/documents/staff_review.php`
`POST /staff/documents/{id}`	`modules/documents/staff_action.php`
`/staff/exam`	`modules/exam/staff_manage.php`
`/staff/exam/slots`	`modules/exam/staff_slots.php`
`/staff/exam/reschedule`	`modules/exam/staff_reschedule.php`
`/staff/exam/cancel-slot`	`modules/exam/staff_cancel_slot.php`
`/staff/exam/export-rooms`	`modules/exam/staff_export_rooms.php`
`/staff/interviews/setup`	`modules/interview/staff_setup.php`
`/staff/interviews/queue`	`modules/interview/staff_queue.php`
`POST /staff/interviews/call-next`	`modules/interview/staff_call_next.php`
`POST /staff/interviews/{id}`	`modules/interview/staff_action.php`
`/staff/interviews/absent`	`modules/interview/staff_absent.php`
`/staff/interviews/cancel-slot`	`modules/interview/staff_cancel_slot.php`
`/staff/results`	`modules/results/staff_manage.php`
`POST /staff/results/bulk`	`modules/results/staff_bulk.php`
`POST /staff/results/auto-release`	`modules/results/staff_auto_release.php`
`POST /staff/results/close`	`modules/results/staff_close_admissions.php`
`POST /staff/results/suggest/{id}`	`modules/results/staff_suggest.php`
`POST /staff/results/{id}`	`modules/results/staff_action.php`
`/staff/settings`	`modules/settings/staff.php`
`/staff/audit-log`	`modules/audit/log.php`

Admin

Path	File
`/admin/dashboard`	`modules/auth/admin/dashboard.php`
`/admin/users`	`modules/settings/admin_users.php`
`/admin/school-year`	`modules/settings/admin_school_year.php`
`/admin/courses`	`modules/settings/admin_courses.php`
`/admin/settings`	`modules/settings/admin.php`
`/admin/results`	`modules/results/admin_export.php`
`/admin/audit-log`	`modules/audit/log.php`

AJAX / API

Path	File
`/api/notifications`	`modules/api/notifications.php`
`/api/exam-autosave`	`modules/api/exam_autosave.php`
`/api/exam-reschedule-request`	`modules/api/exam_reschedule_request.php`
`/api/reschedule-request`	`modules/api/reschedule_request.php`
`/api/applicant-panel`	`modules/api/applicant_panel.php`

> The previous `/api/auto-validate` endpoint was retired in this build.

Database

File	Purpose
`database/schema.sql`	Single-file destructive schema. Creates every table + seeds school settings, departments, courses, passing scores, the seed admin. Idempotent.
`database/seed_users.sql`	Inserts admin, SSO, Deans, Staff, Proctor accounts with pre-set departments.
`database/seed_demo.sql`	Demo applicants spread across the funnel + interview sessions + exam results + notifications. Idempotent.

This document is generated from the source-of-truth code. When in doubt, read the code.